funnelling

Marco Servetto
School of Engineering and Computer Science, Victoria University of Wellington, New Zealand
Colin S. Gordon
School of Computer & Information Sciences, Drexel University, United States of America

Abstract

...

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Dummy keyword

Funding *Colin S. Gordon*: US NSF CISE Award CCF-2007582

Acknowledgements I want to thank ...

1 Introduction

Java includes a useful feature called *method-local inner classes*, which is surprisingly subtle. Like anonymous inner classes, this is a way for a method in a Java program to declare a class whose details are defined locally, entirely within the method, when the identity of the class is not needed outside the method's scope. However, these local classes are named, which allows them to be used with other language features exploiting nominal types. For example, one can cast objects to the method-local class type.

Unfortunately, as Servetto et al. [6] pointed out, these features interact unsoundly with Java generics. In generic methods (or methods of generic classes), a method-local inner class `MyFoo` which extends a type that depends on generic parameters is treated the same regardless of whether it was allocated with the same generic parameters, or different generic parameters. Servetto et al. probed the behaviour of JVM languages which supported the same feature (Scala and Kotlin) and found that these had the same unsoundness problem for some use cases, but did flag concerns in other cases. However this was not systematically investigated.

In this paper we clearly articulate the problem they observed, characterize the key properties that lead to the issue, and demonstrate both dynamic and static approaches to resolve the issue in future languages. The dynamic approach is a specific interpretation of type tests with reified generics, providing guidance to language implementors on a subtle design choice available in that space.

Modern Java: modern Java accepts `instanceof C<Ts>` only when the static type determines `Ts`; with that rule an erased and a reified runtime give the same answer on every well-typed program. The dynamic approach then only concerns languages without that rule (C#-like). Funnelling could be presented as the solution, with reification as a side remark.

The static approach is a general solution for tracking when a nominal object type *declaration* effectively captures generic parameter instantiations from its dynamic context.

Figure 1 demonstrates the issue. Line 3 declares a method-local inner class `MyFoo`, which depends on the generic parameter instantiation for `X` when the method is invoked. The `main` method invokes `m` twice. Line 8 invokes it for `Integers`. Since the `other` argument for that call is not a `MyFoo`, the first invocation returns a `MyFoo` which is a `Foo<Integer>`. This result is then passed back in to the second call to `m` on Line 9. This time, the `instanceof` check

```
1  public class Broken {
2    static <X> Foo<X> m(Object other, final X x){
3      class MyFoo extends Foo<X>{ public X x(){ return x; } }
4      if(!(other instanceof MyFoo)){ return new MyFoo(); }
5      return ((MyFoo)other); // cast passes with no warnings
6    }
7    public static void main(String[] a) {
8      Foo<Integer> r1= Broken.<Integer>m("anything",5);
9      Foo<String> r2= Broken.<String>m(r1,"Hi");
10     System.out.println(r1.x()); //5
11     System.out.println(r2.x()); //Throws ClassCastException: ..
12   }
13 }
14 class Foo<X>{ public X x() {return x();} }
```

Figure 1 Subtly broken Java code

succeeds, because `other` in this case *is* a `MyFoo`, so the method (successfully) casts the `other` argument to `MyFoo`. Since `MyFoo` is declared as extending `Foo<X>`, this is a valid result for the function. Thus the `Foo<Integer>` that was passed as the second argument on Line 9 is returned as a `Foo<String>`.

Clearly this is wrong, because there are now two aliased variables of incompatible types; consequently, the later code fails (we discuss the failure point later). The description of execution above employed some (here, intentionally) sloppy reasoning to parallel the checks in the Java compiler.

- The `instanceof` check could have been statically flagged.
- The cast could have been statically flagged.
- The `instanceof` could have returned false in the second invocation, since it was used on a `MyFoo` from a different generic parameter instantiation
- The case could have failed, again because it was used on a `MyFoo` from a different generic parameter instantiation

Unfortunately two key factors lead to Java accepting this code and only incidentally discovering the problem later:

1. Java implements generics by erasure. So the runtime representation of `MyFoo` doesn't differ across the two calls. No information is kept around to distinguish the two classes of `MyFoo`. This rules out the dynamic solutions.
2. The Java compiler does not treat `MyFoo` as generic. Attempts to cast to, or use `instanceof` with `Foo<Integer>` are statically flagged specifically because erasure means the generic parameters cannot be dynamically checked. But because `MyFoo` *syntactically* lacks generic parameters of its own, the compiler treats it as *non-generic* and allows the `instanceof` and cast without complaint.

Modern Java: precise behaviour (JDK 26): from `Object`, `o instanceof Foo<Integer>` is a compile-time error (since Java 16; before that every non-reifiable `instanceof` target is an error), while the cast `(Foo<Integer>)o` is an unchecked warning; `instanceof` never warns. From a type that determines the argument (`Foo<Integer>` out of a `Bar<List<Integer>>` when `Foo<T> extends Bar<List<T>>`) both are accepted silently. The hole is that `MyFoo` is reifiable, so both forms are accepted with no warning. Item 2 and the list above can be restated in these terms.

```
class Object{};
template<typename X> class Foo : public Object {
    public: X x() { return x(); } };
template<typename X>
Foo<X>* m(Object* other, const X* x) {
    class MyFoo : public Foo<X> { public: X x() { return x; } };
    MyFoo* d = dynamic_cast<MyFoo*>(other);
    if(d == nullptr) { return new MyFoo(); }
    return d;
}
```

Figure 2 C++ equivalent of Figure 1.

Subtler still is the actual cause of the failure. The two `println` calls actually dispatch to different overloads: the first to a version accepting an `Object`, and the second to the more specific overload expecting a `String`. It is in fact the passage of a value from a generic API to the specific overload which causes the Java compiler to insert a `checkcast` in the generated bytecode.

It is tempting to accept this as a quirk of Java, which is already known to have other unsoundness [2]. However this problem is broader in both scope and impact, as we show in the rest of the paper. This issue affects all statically typed languages on the JVM, including Scala and Kotlin. It also propagates to other compilation targets for JVM languages, including Scala Native and GraalVM's Native Image support. cite

In the remainder of this paper:

- We detail the breadth of the problem (surveying forms that use class generics rather than methods generics, closures, and other manifestations), and how it is handled by various existing language implementations
- We develop a core language modelling Java's generics, method-local classes and type tests, and show that typing method-local classes as Java does is unsound.
- We develop a new type system mechanism for systematically tracking would-be implicit dependence of a method-local class on generic parameters, and show that it soundly prevents the problem.

2 Scope of the Problem

This problem arises in essentially all JVM languages, all of which must interact with Java and have therefore chosen to also support method-local classes.

2.1 Generic Reification

A natural suspicion is that this is "simply" a problem with generic erasure, that of course a language that cannot distinguish generic instantiations of a class at runtime will have difficulties of this sort. There is some truth to this: Java is permitting a cast that it cannot actually check due to type erasure. However there is more to it than that: Java will eagerly flag code such as `(List<String>)(new LinkedList<Integer>())`. So the language does intend to warn about uses of uncheckable casts.

Modern Java: `(List<String>)(new LinkedList<Integer>())` is an incompatible-types error, not an unchecked one. The contrast that matters here: an unchecked cast is a warning, an unchecked `instanceof` is an error.

And while reified generics do provide enough information to check casts dynamically, only certain forms of reification make correct casts straightforward.

In a language which implements parametric polymorphism by templating / heterogeneous compilation (C++, C#, or early experiments with generic Java [1]), the check is straightforward: each instantiation of `m` with different generic parameters is compiled separately, resulting in separate class declarations for each instantiation. Consider reinterpreting Figure 1 in C++ as in Figure 2. `gcc` reports for Line 7:

```
<source>:7:16: error: cannot 'dynamic_cast' 'other' (of type 'class Object*')
to type 'class m<Integer>(Object*, const Integer*)::MyFoo*' (source type is not polymorphic)
    7 |     MyFoo* d = dynamic_cast<MyFoo*>(other);
      |                ^~~~~~~~~~~~~~~~~~~~~~~~~~~
```

not perfect, because this is C++ giving a static warning, not catching the error dynamically...

Notice that the error message uses the instantiation of the method signature as a scope for a declaration of `MyFoo`. But heterogeneous compilation has subtle costs and wide-ranging consequences for language design, not just implementation, and complex performance consequences [5, 4]. Even if this does fix the problem, it is not universally appropriate.

An alternative, common conception of generic reification is to simply save the type parameters along with the single implementations of classes, for it to be checked when needed. But note again that `MyFoo` has no generic parameters of its own. So a shallow check with this variant of reified generics is insufficient — each `MyFoo` would need to contain a representation of, or reference to, its parent type's generic parameters. And because this phenomenon is not limited to a single step in the inheritance hierarchy, the cast must traverse and compare a substantial amount of data — not a mere comparison of virtual table pointers. This again has multiple implementation approaches, neither ideal (chains of parent type instantiations hurt cache locality, inlining data can bloat small instances).

So there remains a need to articulate a solution to this problem where generics are implemented via erasure.

Modern Java: under erasure, Java's checked-cast rule is already enough once the dependency of `MyFoo` on `X` is visible in its type, and that is what funnelling provides. This paragraph can lead straight into Section 3.

ah, we need to check Flix, where maybe we can cook up something roughly like this?

3 A Core Language for Funnelling

Expressions	e	::=	$x \mid e.\texttt{<}Ts\texttt{>}m(es) \mid e.[T]\texttt{<}Ts\texttt{>}m(es) \mid e.f \mid \texttt{new}\ C\texttt{<}Ts\texttt{>}(es)$
			$\mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts'\texttt{>}(es)\{Ms\}$
			$\mid \texttt{if}\ (e_0\ \texttt{instanceof}\ T\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \}$
Types	T	::=	$C\texttt{<}Ts\texttt{>} \mid X$
Funnels	F	::=	$G \mid X\ \texttt{funnel}$
Ground Types	G	::=	$C\texttt{<}Gs\texttt{>}$
Type Env.	Γ	::=	$T_1\ x_1, \ldots, T_n\ x_n$
Bounds	Δ	::=	$X_1\ \texttt{extends}\ T_1, \ldots, X_n\ \texttt{extends}\ T_n$
Declaration	Dec	::=	$\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ Ss\ \}$
(Field) Slots	S	::=	$T\ f\texttt{;}$
Methods	M	::=	$\texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \{\ \texttt{return}\ e\texttt{;}\ \}$
Class Decl.	K	::=	$\text{Dec} \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\}$

Figure 3 Syntax

A program is a class table Decls and a main expression e. We require the following well-formedness conditions:

(A1) Class names are unique across top-level and local classes.
(A2) Field names are unique across all classes.
(A3) In a class, top-level or local, all method names are disjoint.
(A4) In a method, all the parameters $x_1 \ldots x_n$ are disjoint.
(A5) In a Δ, all the X are disjoint.
(A6) In a list of funnels Fs, all the funnelled type variables are disjoint.
(A7) There is no shadowing of X or x, except for `this`. No parameter and no variable of an `if` is `this`.
(A8) Decls contains `class Object<> extends Object<> { }`, and the chain of superclasses of every other top-level class reaches `Object`.

Well-formedness requirements A1-A8 are common requirements, such as uniqueness of names. To simplify scoping, no shadowing is allowed. A funnel X `funnel` does not bind X: it uses the type variable X in scope. A method of a local class binds `this`, as in Java; this is the only form of shadowing. Requirement A6 makes the substitutions of Figure 10 well defined when their domain is a list of funnels. Requirement A8 makes inheritance acyclic, except for `Object`, which is its own superclass. Thus rule SUB-INHERIT can be applied to `Object<>` forever, and rules SIMPLEFIELDSUPER and SEARCHSUPER reduce a field access or a call on an `Object` to itself. This is harmless: our rules are not meant to be read as algorithms, and a well-typed expression never accesses a field or calls a method of `Object`.

Well-formedness requirements B1-B3 below are about local classes:

(B1) The main expression contains no local classes.
(B2) Local classes cannot be extended: the superclass of every class, top-level or local, is a top-level class.
(B3) In a local class $\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\}$, every funnelled type variable is in scope.

Requirement B1 makes every local class occur in a method body, so that the type variables in scope where it is written are defined. Requirements B2 and B3 are enforced by the type system, but we call them out because they are explicit design choices.

$$\boxed{K \text{ declared}}$$

$$\text{DECL-TOP}\quad \frac{\mathsf{Dec} \in \mathsf{Decls}}{\mathsf{Dec\ declared}}$$

$$\text{DECL-IN-CLASS}\quad \frac{\begin{array}{c}\texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ Ms\ \texttt{_\}}\ \text{declared} \\ \texttt{<_>_}\ m\texttt{(_)}\ \texttt{\{}\ \texttt{return}\ e\texttt{;}\ \texttt{\}} \in Ms \\ \texttt{new}\ C_0\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms'\texttt{\}}\ \text{occurs in}\ e\end{array}}{\texttt{new}\ C_0\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms'\texttt{\}}\ \text{declared}}$$

$$\text{DECL-IN-LOCAL}\quad \frac{\begin{array}{c}\texttt{new}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>(_)\{}Ms\texttt{\}}\ \text{declared} \\ \texttt{<_>_}\ m\texttt{(_)}\ \texttt{\{}\ \texttt{return}\ e\texttt{;}\ \texttt{\}} \in Ms \\ \texttt{new}\ C_0\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms'\texttt{\}}\ \text{occurs in}\ e\end{array}}{\texttt{new}\ C_0\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms'\texttt{\}}\ \text{declared}}$$

$$\boxed{\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts'} \qquad \boxed{\mathsf{scope}(\Delta; Fs; Ts) = \Delta'; Ts'}$$

$$\begin{array}{rcl}\mathsf{scope}(C\texttt{<}Ts\texttt{>}) & = & \Delta; Ts \\ \quad \text{where}\ \texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{\ _\ _\ \}} \in \mathsf{Decls} & & \\ \mathsf{scope}(C_0\texttt{<}Ts\texttt{>}) & = & \mathsf{scope}(\Delta', \Delta''; Fs; Ts) \\ \quad \text{where}\ \texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ Ms\ \texttt{_\}}\ \text{declared or}\ \texttt{new}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>(_)\{}Ms\texttt{\}}\ \text{declared} & & \\ \qquad \texttt{<}\Delta''\texttt{>_}\ m\texttt{(_)}\ \texttt{\{}\ \texttt{return}\ e\texttt{;}\ \texttt{\}} \in Ms & & \\ \qquad \texttt{new}\ C_0\texttt{<}Fs\texttt{>}\ \texttt{extends}\ \texttt{_<_>(_)\{_\}}\ \text{occurs in}\ e & & \\ \qquad \mathsf{scope}(\mathsf{generic}(C)) = \Delta'; _ & & \end{array}$$

$$\begin{array}{rcl}\mathsf{scope}(\Delta; \emptyset; \emptyset) & = & \emptyset; \emptyset \\ \mathsf{scope}(\Delta; X\ \texttt{funnel}\ Fs; T\ Ts) & = & (X\ \texttt{extends}\ \Delta(X); T) \cdot \mathsf{scope}(\Delta; Fs; Ts) \\ \mathsf{scope}(\Delta; G\ Fs; G\ Ts) & = & \mathsf{scope}(\Delta; Fs; Ts) \\ (\Delta; Ts) \cdot (\Delta'; Ts') & = & \Delta, \Delta'; Ts\ Ts'\end{array}$$

$$\boxed{\mathsf{generic}(C) = T} \qquad \boxed{|F| = T}$$

$$\begin{array}{rcl}\mathsf{generic}(C) & = & C\texttt{<}\mathrm{dom}(\Delta)\texttt{>} \\ \quad \text{where}\ \texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{\ _\ _\ \}} \in \mathsf{Decls} & & \\ \mathsf{generic}(C) & = & C\texttt{<}|Fs|\texttt{>} \\ \quad \text{where}\ \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ \texttt{_<_>(_)\{_\}}\ \text{declared} & & \\ |X\ \texttt{funnel}| & = & X \\ |G| & = & G\end{array}$$

Figure 4 Declarations

By well-formedness requirements A1 and B1, a class name C identifies at most one K with K declared. For a class type $C\texttt{<}Ts\texttt{>}$, $\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta_0; Ts''$ gives the bounds Δ_0 of the type variables that the methods of C see, without the type variables of the methods themselves. It also gives the types Ts'' that the arguments Ts assign to the type variables in Δ_0. For a top-level class, Δ_0 is the bounds of the class and $Ts'' = Ts$. For a local class

C_0 with funnels Fs, the bounds Δ', Δ'' are the bounds of the type variables in scope where C_0 is written. For a type variable X, $\Delta(X)$ is the bound of X in Δ, and $\mathrm{cod}(\Delta)$ is the sequence of the bounds in Δ. The function $\mathsf{scope}(\Delta; Fs; Ts)$ goes through the funnels Fs and the arguments Ts together, one position at a time, where $\emptyset$ is the empty sequence. At the position of a funnel X `funnel`, it adds X `extends` $\Delta(X)$ to the bounds and the argument to the types. At the position of a ground funnel G, the argument must be G itself, and nothing is added. The operator $\cdot$ concatenates two results of scope, component by component. An expression e' occurs in e when $e = \mathcal{C}[e']$, for a context $\mathcal{C}$ of Figure 13. Contexts $\mathcal{C}$ do not enter the methods of a local class; thus $\mathsf{scope}(C\texttt{<}Ts\texttt{>})$ has at most one value. The reduction of Figure 5 uses the values v and the evaluation contexts $\mathcal{E}$ of Figure 13.

Rules simpleFound and innerFound use the method $\mathsf{fresh}(M)$: the method M where every type variable and every variable bound in M is renamed into a fresh name, that is a name occurring nowhere in the program and in the reduced expression. The type variables and variables bound in M are the type variables of its signature, its parameters, and, inside its body, the variable of every `if` and the type variables and parameters of every method of a local class. The variable `this` is never renamed. Renaming is needed because a value of a local class carries the code of its methods. For example, consider a local class `L` whose method `m` has a parameter `x` and calls `m` on a captured value of class `L`. The body of a call of `m` then contains the captured value, and the captured value contains a copy of the method `m` binding `x` again. Without renaming, rule innerFound would replace `x` also in the method `m` of the captured value. With renaming, every name bound inside a value is fresh: only values of local classes carry code, by requirement B1 local classes occur only in method bodies, and a method body enters the reduced expression only through rules simpleFound and innerFound, renamed by fresh. Outside of values, the names bound inside the reduced expression are not shadowed, by requirement A7. Thus a substitution never reaches a type variable or a variable bound inside the substituted expression.

$$\boxed{e \to e'}$$

simpleFieldFound

$$\frac{\texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ _\texttt{<_>}\ \{\ _\ T_1\ f_1\texttt{;}\ \ldots T_n\ f_n\texttt{;}\ \} \in \mathsf{Decls} \qquad f_i \in f_1 \ldots f_n}{\texttt{new}\ C\texttt{<}Gs\texttt{>(}v_1 \ldots v_n\ vs\texttt{)}\,.\,f_i \to v_i}$$

simpleFieldSuper

$$\frac{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ _\ T_1\ f_1\texttt{;}\ \ldots T_n\ f_n\texttt{;}\ \} \in \mathsf{Decls} \qquad f \notin f_1 \ldots f_n}{\texttt{new}\ C\texttt{<}Gs\texttt{>(}v_1 \ldots v_n\ vs\texttt{)}\,.\,f \to \texttt{new}\ C'\texttt{<}Ts[\Delta = Gs]\texttt{>(}vs\texttt{)}\,.\,f}$$

innerFieldFound

$$\frac{}{\texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{)\{}Ms\texttt{\}}\,.\,f \to \texttt{new}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{)}\,.\,f}$$

ifThen

$$\frac{v \in \{\texttt{new}\ C\texttt{<}Gs\texttt{>(_)}, \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ _\texttt{<_>(_)\{_\}}\} \qquad \emptyset \vdash C\texttt{<}Gs\texttt{>} <: C'\texttt{<_>}}{\texttt{if (}v\ \texttt{instanceof}\ C'\texttt{<}Ts\texttt{>}\ x\texttt{)}\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \} \to e_1[x = v]}$$

ifElse

$$\frac{v \in \{\texttt{new}\ C\texttt{<}Gs\texttt{>(_)}, \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ _\texttt{<_>(_)\{_\}}\} \qquad \emptyset \not\vdash C\texttt{<}Gs\texttt{>} <: C'\texttt{<_>}}{\texttt{if (}v\ \texttt{instanceof}\ C'\texttt{<}Ts\texttt{>}\ x\texttt{)}\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \} \to e_2}$$

simpleAddSuper

$$\frac{v = \texttt{new}\ C\texttt{<}Gs\texttt{>(_)}}{v\texttt{.<}Gs'\texttt{>}m\texttt{(}vs\texttt{)} \to v\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m\texttt{(}vs\texttt{)}}$$

simpleFound

$$\frac{\begin{array}{c}\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ _\texttt{<_>}\ \{\ Ms\ _\ \} \in \mathsf{Decls} \\ M \in Ms \qquad \mathsf{fresh}(M) = \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots T_n\ x_n\texttt{)}\ \{\ \texttt{return}\ e_0\texttt{;}\ \} \\ e = e_0[\Delta, \Delta' = Gs, Gs'][\texttt{this}, x_1 \ldots x_n = v_0 \ldots v_n]\end{array}}{v_0\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m\texttt{(}v_1 \ldots v_n\texttt{)} \to e}$$

innerFound

$$\frac{\begin{array}{c}v_0 = \texttt{new}\ _\texttt{<_>}\ \texttt{extends}\ _\texttt{<_>(_)\{}Ms\texttt{\}} \\ M \in Ms \qquad \mathsf{fresh}(M) = \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots T_n\ x_n\texttt{)}\ \{\ \texttt{return}\ e_0\texttt{;}\ \} \\ e = e_0[\Delta' = Gs'][\texttt{this}, x_1 \ldots x_n = v_0 \ldots v_n]\end{array}}{v_0\texttt{.<}Gs'\texttt{>}m\texttt{(}v_1 \ldots v_n\texttt{)} \to e}$$

innerSuper

$$\frac{v_0 = \texttt{new}\ _\texttt{<_>}\ \texttt{extends}\ C\texttt{<}Gs\texttt{>(_)\{}Ms\texttt{\}} \qquad m \notin Ms}{v_0\texttt{.<}Gs'\texttt{>}m\texttt{(}v_1 \ldots v_n\texttt{)} \to v_0\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m\texttt{(}v_1 \ldots v_n\texttt{)}}$$

searchSuper

$$\frac{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ _\ \} \in \mathsf{Decls} \qquad m \notin Ms}{v\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m\texttt{(}vs\texttt{)} \to v\texttt{.[}C'\texttt{<}Ts[\Delta = Gs]\texttt{>]<}Gs'\texttt{>}m\texttt{(}vs\texttt{)}}$$

ctx

$$\frac{e \to e'}{\mathcal{E}[e] \to \mathcal{E}[e']}$$

Figure 5 Operational Semantics

$$\boxed{\Delta;\Gamma \vdash e : T}$$

SUBS
$$\frac{\Delta;\Gamma \vdash e : T_1 \qquad \Delta \vdash T_1 <: T_2 \qquad \Delta \vdash T_2 \ \mathrm{Ok}}{\Delta;\Gamma \vdash e : T_2}$$

VAR
$$\frac{}{\Delta;\Gamma \vdash x : \Gamma(x)}$$

NEW-OBJECT
$$\frac{}{\Delta;\Gamma \vdash \texttt{new Object<>()} : \texttt{Object<>}}$$

DECLFIELD
$$\frac{\Delta;\Gamma \vdash e : C\texttt{<}Ts\texttt{>} \qquad \texttt{class } C\texttt{<}\Delta'\texttt{> extends _<_> \{ _ } Ss \texttt{ }\} \in \mathsf{Decls} \qquad T\ f\texttt{;} \in Ss}{\Delta;\Gamma \vdash e.f : T[\Delta' = Ts]}$$

CALL-TOP
$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_0 : C_0\texttt{<}Ts'\texttt{>} \qquad \Delta \vdash Ts'' \ \mathrm{Ok} \\ \texttt{class } C_0\texttt{<}\Delta_0\texttt{> extends _<_> \{ } Ms \texttt{ _ }\} \ \mathsf{declared} \\ \texttt{<}\Delta_1\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots T_n\ x_n\texttt{) \{ _ }\} \in Ms \\ \Delta \vdash Ts'' <: \mathrm{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts', Ts''] \\ \Delta;\Gamma \vdash e_1 \ldots e_n : T_1 \ldots T_n[\Delta_0, \Delta_1 = Ts', Ts'']\end{array}}{\Delta;\Gamma \vdash e_0.\texttt{<}Ts''\texttt{>}m\texttt{(}e_1, \ldots, e_n\texttt{)} : T_0[\Delta_0, \Delta_1 = Ts', Ts'']}$$

CALL-LOCAL
$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_0 : C_0\texttt{<}Ts'\texttt{>} \qquad \Delta \vdash Ts'' \ \mathrm{Ok} \\ \texttt{new } C_0\texttt{<}Fs\texttt{> extends _<_>(_)\{}Ms\texttt{\}} \ \mathsf{declared} \\ \texttt{<}\Delta_1\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots T_n\ x_n\texttt{) \{ _ }\} \in Ms \\ \Delta \vdash Ts'' <: \mathrm{cod}(\Delta_1)[Fs, \Delta_1 = Ts', Ts''] \\ \Delta;\Gamma \vdash e_1 \ldots e_n : T_1 \ldots T_n[Fs, \Delta_1 = Ts', Ts'']\end{array}}{\Delta;\Gamma \vdash e_0.\texttt{<}Ts''\texttt{>}m\texttt{(}e_1, \ldots, e_n\texttt{)} : T_0[Fs, \Delta_1 = Ts', Ts'']}$$

CALL-LB-RUNTIME
$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_0 : C\texttt{<}Ts'''\texttt{>} \qquad \Delta \vdash C\texttt{<}Ts'''\texttt{>} \ \mathrm{Ok} \qquad \Delta \vdash Ts'' \ \mathrm{Ok} \\ \texttt{class } C\texttt{<_> extends _<_> \{ _ _ }\} \in \mathsf{Decls} \qquad \Delta \vdash C\texttt{<}Ts'''\texttt{>} <: C_0\texttt{<}Ts'\texttt{>} \\ \texttt{class } C_0\texttt{<}\Delta_0\texttt{> extends _<_> \{ } Ms \texttt{ _ }\} \ \mathsf{declared} \qquad \texttt{<}\Delta_1\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots T_n\ x_n\texttt{) \{ _ }\} \in Ms \\ \Delta \vdash Ts'' <: \mathrm{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts', Ts''] \\ \Delta;\Gamma \vdash e_1 \ldots e_n : T_1 \ldots T_n[\Delta_0, \Delta_1 = Ts', Ts'']\end{array}}{\Delta;\Gamma \vdash e_0.\texttt{[}C\texttt{<}Ts'''\texttt{>]<}Ts''\texttt{>}m\texttt{(}e_1, \ldots, e_n\texttt{)} : T_0[\Delta_0, \Delta_1 = Ts', Ts'']}$$

NEW-TOP
$$\frac{\begin{array}{c}\texttt{class } C\texttt{<}\Delta'\texttt{> extends } C'\texttt{<}Ts'\texttt{> \{ _ }T_1\ f_1\texttt{; } \ldots T_n\ f_n\texttt{; }\} \ \mathsf{declared} \\ \Delta;\Gamma \vdash \texttt{new } C'\texttt{<}Ts'[\Delta' = Ts]\texttt{>(}es\texttt{)} : _ \qquad \Delta;\Gamma \vdash e_1 \ldots e_n : T_1 \ldots T_n[\Delta' = Ts] \qquad \Delta \vdash C\texttt{<}Ts\texttt{>} \ \mathrm{Ok}\end{array}}{\Delta;\Gamma \vdash \texttt{new } C\texttt{<}Ts\texttt{>(}e_1 \ldots e_n\ es\texttt{)} : C\texttt{<}Ts\texttt{>}}$$

NEW-LOCAL
$$\frac{\begin{array}{c}\mathsf{scope}(\Delta; Fs; |Fs|) = \Delta'; _ \qquad \mathsf{FTV}(\mathrm{cod}(\Delta')) \subseteq \mathrm{dom}(\Delta') \qquad \Delta' \vdash C\texttt{<}|Fs|\texttt{>} \ \mathrm{Ok} \\ \Delta;\Gamma \vdash \texttt{new } C'\texttt{<}Ts\texttt{>(}es\texttt{)} : _ \qquad \Delta' \vdash C'\texttt{<}Ts\texttt{>} \ \mathrm{Ok} \\ \Gamma' = \{T\ x \mid T\ x \in \Gamma \wedge x \neq \texttt{this} \wedge \mathsf{FTV}(T) \subseteq \mathsf{FTV}(|Fs|)\} \\ \Delta';\Gamma', C\texttt{<}|Fs|\texttt{> this} \vdash Ms \ \mathrm{Ok} \\ \forall C'', Ts'.\ \Delta' \vdash C'\texttt{<}Ts\texttt{>} <: C''\texttt{<}Ts'\texttt{>} \Rightarrow \mathsf{overrideOk}(C''\texttt{<}Ts'\texttt{>}, Ms)\end{array}}{\Delta;\Gamma \vdash \texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}} : C\texttt{<}|Fs|\texttt{>}}$$

IF-INSTANCEOF
$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_0 : T_0 \qquad \Delta \vdash T \ \mathrm{Ok} \qquad \Delta \vdash T_0 \rightsquigarrow T \\ \Delta;\Gamma, T\ x \vdash e_1 : T' \qquad \Delta;\Gamma \vdash e_2 : T'\end{array}}{\Delta;\Gamma \vdash \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{ }e_1\texttt{ \} else \{ }e_2\texttt{ \}} : T'}$$

Figure 6 Typing

$$\boxed{\vdash \mathsf{Decls}\ e\ \mathrm{Ok}}$$

PROG-OK
$$\frac{\forall \mathsf{Dec} \in \mathsf{Decls}.\ \mathsf{Dec}\ \mathrm{Ok} \qquad \emptyset;\emptyset \vdash e : _}{\vdash \mathsf{Decls}\ e\ \mathrm{Ok}}$$

$$\boxed{\mathsf{Dec}\ \mathrm{Ok}}$$

CLASS-OK
$$\frac{\begin{array}{c}\Delta \vdash \mathrm{cod}(\Delta)\ \mathrm{Ok} \qquad \forall T\ f\texttt{;} \in Ss.\ \Delta \vdash T\ \mathrm{Ok} \\ \texttt{class}\ C'\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{\ _\ _\ \}} \in \mathsf{Decls} \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>}\ \mathrm{Ok} \\ \Delta; C\texttt{<}\mathrm{dom}(\Delta)\texttt{>}\ \texttt{this} \vdash Ms\ \mathrm{Ok} \\ \forall C'', Ts'.\ \Delta \vdash C'\texttt{<}Ts\texttt{>} <: C''\texttt{<}Ts'\texttt{>} \Rightarrow \mathsf{overrideOk}(C''\texttt{<}Ts'\texttt{>}, Ms)\end{array}}{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ Ms\ Ss\ \texttt{\}}\ \mathrm{Ok}}$$

$$\boxed{\Delta;\Gamma \vdash M\ \mathrm{Ok}}$$

METHOD-OK
$$\frac{\begin{array}{c}\Delta, \Delta' \vdash \mathrm{cod}(\Delta')\ \mathrm{Ok} \qquad \Delta, \Delta' \vdash T_0 \ldots T_n\ \mathrm{Ok} \\ \Delta, \Delta'; \Gamma, T_1\ x_1, \ldots, T_n\ x_n \vdash e : T_0\end{array}}{\Delta;\Gamma \vdash \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{)}\ \texttt{\{}\ \texttt{return}\ e\texttt{;}\ \texttt{\}}\ \mathrm{Ok}}$$

$$\boxed{\mathsf{overrideOk}(T, M)}$$

OVERRIDE-NEW
$$\frac{\texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ Ms\ \texttt{_}\ \texttt{\}} \in \mathsf{Decls} \qquad m \notin Ms}{\mathsf{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<_>_}\ m\texttt{(_)}\ \texttt{\{\ _\ \}})}$$

OVERRIDE-SAME
$$\frac{\begin{array}{c}\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ Ms\ \texttt{_}\ \texttt{\}} \in \mathsf{Decls} \\ \texttt{<}\Delta''\texttt{>}T_0''\ m\texttt{(}T_1''\ x_1', \ldots, T_n''\ x_n'\texttt{)}\ \texttt{\{\ _\ \}} \in Ms \\ \mathrm{cod}(\Delta'')[\Delta, \Delta'' = Ts, \mathrm{dom}(\Delta')] = \mathrm{cod}(\Delta') \\ T_0'' \ldots T_n''[\Delta, \Delta'' = Ts, \mathrm{dom}(\Delta')] = T_0' \ldots T_n'\end{array}}{\mathsf{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<}\Delta'\texttt{>}T_0'\ m\texttt{(}T_1'\ x_1, \ldots, T_n'\ x_n\texttt{)}\ \texttt{\{\ _\ \}})}$$

Figure 7 Typing of programs, classes and methods

Rule CLASS-OK types the methods of a top-level class with `this` of type $C\texttt{<}\mathrm{dom}(\Delta)\texttt{>}$. A method that a class declares again must keep the signature it has in every superclass, up to the names of its own type variables: CALL-TOP reads the signature from the class chosen by subsumption, while the reduction runs the method of the class of the receiver.

$$\boxed{\Delta \vdash T \text{ Ok}}$$

$$\text{WF-VAR}\ \frac{X \texttt{ extends } _ \in \Delta}{\Delta \vdash X \text{ Ok}} \qquad \text{WF-CLASS}\ \frac{\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta_0; Ts'' \qquad \Delta \vdash Ts \text{ Ok} \qquad \Delta \vdash Ts'' <: \mathrm{cod}(\Delta_0)[\Delta_0 = Ts'']}{\Delta \vdash C\texttt{<}Ts\texttt{>} \text{ Ok}}$$

$$\boxed{\Delta \vdash T <: T'}$$

$$\text{SUB-REFL}\ \frac{}{\Delta \vdash T <: T} \qquad \text{SUB-INHERIT}\ \frac{\texttt{class } C\texttt{<}\Delta_0\texttt{>} \texttt{ extends } C'\texttt{<}Ts'\texttt{>} \texttt{ \{ _ _ \}} \in \mathsf{Decls} \qquad \Delta \vdash C'\texttt{<}Ts'\texttt{>}[\Delta_0 = Ts] <: T''}{\Delta \vdash C\texttt{<}Ts\texttt{>} <: T''}$$

$$\text{SUB-INHERIT-LOCAL}\ \frac{\texttt{new } C\texttt{<}Fs\texttt{>} \texttt{ extends } C'\texttt{<}Ts'\texttt{>(_)\{_\}} \text{ declared} \qquad \Delta \vdash C'\texttt{<}Ts'\texttt{>}[Fs = Ts] <: T''}{\Delta \vdash C\texttt{<}Ts\texttt{>} <: T''} \qquad \text{SUB-VAR}\ \frac{X \texttt{ extends } T \in \Delta \qquad \Delta \vdash T <: T'}{\Delta \vdash X <: T'}$$

Figure 8 Kinding and Subtyping

Rule WF-CLASS kinds both top-level and local classes, including a local class written in a method of another local class. Every type in Ts'' is checked against the bound of its type variable in Δ_0. A funnelled type variable that is not in scope where its local class C is written leaves $\mathsf{scope}(C\texttt{<}Ts\texttt{>})$ undefined; thus no type $C\texttt{<}Ts\texttt{>}$ is well kinded.

$$\boxed{\Delta \vdash T \rightsquigarrow T'}$$

$$\text{CAST-UP}\ \frac{\Delta \vdash T <: T'}{\Delta \vdash T \rightsquigarrow T'} \qquad \text{CAST-VAR}\ \frac{X \texttt{ extends } T \in \Delta \qquad \Delta \vdash T \rightsquigarrow T'}{\Delta \vdash X \rightsquigarrow T'}$$

$$\text{CAST-DOWN}\ \frac{\begin{array}{ll}\Delta \vdash C\texttt{<}Ts\texttt{>} <: C'\texttt{<}Ts_0\texttt{>} & \mathrm{generic}(C) = C\texttt{<}Ts''\texttt{>} \\ \emptyset \vdash C\texttt{<}Ts''\texttt{>} <: C'\texttt{<}Ts'\texttt{>} & \mathsf{FTV}(Ts'') \subseteq \mathsf{FTV}(Ts')\end{array}}{\Delta \vdash C'\texttt{<}Ts_0\texttt{>} \rightsquigarrow C\texttt{<}Ts\texttt{>}}$$

Figure 9 Checked Casts

$$
\begin{array}{rcl}
x[Xs = Ts] & = & x \\
e.\texttt{<}Ts'\texttt{>}m(es')[Xs = Ts] & = & e[Xs = Ts].\texttt{<}Ts'[Xs = Ts]\texttt{>}m(es'[Xs = Ts]) \\
e.\texttt{[}T'\texttt{]<}Ts''\texttt{>}m(es')[Xs = Ts] & = & e[Xs = Ts].\texttt{[}T'[Xs = Ts]\texttt{]<}Ts''[Xs = Ts]\texttt{>} \\
 & & m(es'[Xs = Ts]) \\
e.f[Xs = Ts] & = & e[Xs = Ts].f \\
\texttt{new } C\texttt{<}Ts'\texttt{>(}es\texttt{)}[Xs = Ts] & = & \texttt{new } C\texttt{<}Ts'[Xs = Ts]\texttt{>(}es[Xs = Ts]\texttt{)} \\
\texttt{if (}e_0\texttt{ instanceof }T'\ x\texttt{)} & = & \texttt{if (}e_0[Xs = Ts]\texttt{ instanceof }T'[Xs = Ts]\ x\texttt{)} \\
\texttt{\{ }e_1\texttt{ \} else \{ }e_2\texttt{ \}}\ [Xs = Ts] & & \texttt{\{ }e_1[Xs = Ts]\texttt{ \} else \{ }e_2[Xs = Ts]\texttt{ \}} \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts'\texttt{>} & = & \texttt{new } C\texttt{<}Fs[Xs = Ts]\texttt{> extends } C'\texttt{<}Ts'[Xs = Ts]\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [Xs = Ts] & & \texttt{(}es[Xs = Ts]\texttt{)\{}Ms[Xs = Ts]\texttt{\}} \\
X'\texttt{ funnel}[Xs = Ts] & = & X'\texttt{ funnel} \quad \text{if } X' \notin Xs \\
X_i\texttt{ funnel}[X_1 \ldots X_n = T_1 \ldots T_n] & = & T_i \\
G[Xs = Ts] & = & G \\
\texttt{<}\Delta\texttt{>}T_0'\ m(T_1'\ x_1, \ldots, T_k'\ x_k) & = & \texttt{<}\Delta[Xs = Ts]\texttt{>}T_0'[Xs = Ts]\ m(T_1'[Xs = Ts]\ x_1, \ldots, T_k'[Xs = Ts]\ x_k) \\
\texttt{\{ return }e\texttt{; \}}\ [Xs = Ts] & & \texttt{\{ return }e[Xs = Ts]\texttt{; \}} \\
(X'\texttt{ extends }T')[Xs = Ts] & = & X'\texttt{ extends }T'[Xs = Ts] \\
C\texttt{<}Ts'\texttt{>}[Xs = Ts] & = & C\texttt{<}Ts'[Xs = Ts]\texttt{>} \\
X'[Xs = Ts] & = & X' \quad \text{if } X' \notin Xs \\
X_i[X_1 \ldots X_n = T_1 \ldots T_n] & = & T_i
\end{array}
$$

Figure 10 Substitution of type variables

Substitution replaces every occurrence of a type variable, *including* the ones in the method signatures of a method-local class. Substitution does not rename the type variables it meets. No renaming is needed: the substituted types are ground, and the domain of the substitution never contains a type variable bound inside the substituted term, by requirement A7 in the program and by fresh during reduction. In the domain of a substitution, bounds Δ stand for $\mathrm{dom}(\Delta)$, and funnels Fs stand for their funnelled type variables: $T[F_1 \ldots F_n = T_1 \ldots T_n]$ replaces X_i with T_i for every $F_i = X_i$ `funnel`, and ignores every $F_i = G_i$. Substitution of type variables into expressions happens only during reduction, where the substituted types are ground. Thus X_i `funnel`$[X_1 \ldots X_n = T_1 \ldots T_n] = T_i$ is a ground type G, and the result is again a funnel.

$$
\begin{array}{rcl}
x[xs = vs] & = & x \quad \text{if } x \notin xs \\
x_i[x_1 \ldots x_n = v_1 \ldots v_n] & = & v_i \\
e.\texttt{<}Ts\texttt{>}m(es')[xs = vs] & = & e[xs = vs].\texttt{<}Ts\texttt{>}m(es'[xs = vs]) \\
e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es')[xs = vs] & = & e[xs = vs].\texttt{[}T\texttt{]<}Ts\texttt{>}m(es'[xs = vs]) \\
e.f[xs = vs] & = & e[xs = vs].f \\
\texttt{new } C\texttt{<}Ts\texttt{>(}es\texttt{)}[xs = vs] & = & \texttt{new } C\texttt{<}Ts\texttt{>(}es[xs = vs]\texttt{)} \\
\texttt{if (}e_0\texttt{ instanceof }T\ x'\texttt{)} & = & \texttt{if (}e_0[xs = vs]\texttt{ instanceof }T\ x'\texttt{)} \\
\texttt{\{ }e_1\texttt{ \} else \{ }e_2\texttt{ \}}\ [xs = vs] & & \texttt{\{ }e_1[xs = vs]\texttt{ \} else \{ }e_2[xs = vs]\texttt{ \}} \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} & = & \texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [xs = vs] & & \texttt{(}es[xs = vs]\texttt{)\{}Ms[xs = vs]\texttt{\}} \\
 & & \quad \text{if } \texttt{this} \notin xs \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} & = & \texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [xs, \texttt{this}, xs' = vs, v, vs'] & & \texttt{(}es[xs, \texttt{this}, xs' = vs, v, vs']\texttt{)\{}Ms[xs, xs' = vs, vs']\texttt{\}} \\
\texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x_1', \ldots, T_k\ x_k') & = & \texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x_1', \ldots, T_k\ x_k') \\
\texttt{\{ return }e\texttt{; \}}\ [xs = vs] & & \texttt{\{ return }e[xs = vs]\texttt{; \}}
\end{array}
$$

Figure 11 Substitution of variables

A method-local class binds `this`, as in Java: inside the methods of a method-local class, `this` is the local object, and the outer `this` is hidden. Thus substitution of variables enters the methods of a method-local class for every variable except `this`. The variable x' of an `if` and the parameters $x'_1 \dots x'_k$ of a method are never in xs, by requirement A7 in the program and by fresh during reduction; so no renaming is needed.

...

The rule NEW-LOCAL is the heart of funneling, including subtle management of type environments. The management of type environments in NEW-LOCAL is subtle. The rule constructs a restricted set of type bounds and variable bindings reflecting only the explicitly-funnelled type variables: Δ' includes only the explicitly-funnelled type variables, and Γ' keeps only the variables of Γ, other than `this`, whose types mention only type variables of the result type $C\texttt{<}|Fs|\texttt{>}$ (which must be well-kinded under Δ'). The bounds Δ' are computed by $\mathsf{scope}(\Delta; Fs; |Fs|)$, which is undefined when a funnelled type variable is not in Δ. However, the rule still uses the *full* ambient environments for checking certain subexpressions. This is because some $e \in es$ may initially contain uses of *non*-funnelled type variables (e.g., an explicit call), but will reduce to a value which no longer depends on such type variables. This is why the premise $\Delta; \Gamma \vdash \texttt{new}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)} : _$ checks the effective supertype instance under $\Delta; \Gamma$, but the premise $\Delta' \vdash C'\texttt{<}Ts\texttt{>}$ Ok ensures the resulting type of that expression satisfies the funnelling constraints by checking kinding under the more restrictive Δ'.

NEW-LOCAL-JAVA

$$\frac{\Delta; \Gamma \vdash \texttt{new}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)} : _ \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>}\ \text{Ok} \qquad \Delta; \Gamma \setminus \texttt{this}, C\texttt{<>}\ \texttt{this} \vdash Ms\ \text{Ok} \qquad \forall C'', Ts'.\ \Delta \vdash C'\texttt{<}Ts\texttt{>} <: C''\texttt{<}Ts'\texttt{>} \Rightarrow \mathsf{overrideOk}(C''\texttt{<}Ts'\texttt{>}, Ms)}{\Delta; \Gamma \vdash \texttt{new}\ C\texttt{<>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}} : C\texttt{<>}}$$

Figure 12 Java typing of local classes

Rule NEW-LOCAL-JAVA of Figure 12 types local classes as Java does. A local class has no funnels, and its methods are typed under the full Δ and Γ in scope, except for `this`. Replacing rule NEW-LOCAL with rule NEW-LOCAL-JAVA makes our system unsound for the same reason Java is unsound. Consider the method `mk` below, the method `m` of Figure 1 in our syntax:

```
<X> Foo<X> mk(Object<> other, X x){ return
  if (other instanceof MyFoo<> w){ w }
  else { new MyFoo<> extends Foo<X>(){ X x(){ return x; } } }; }
```

Rule NEW-LOCAL-JAVA gives to the object created by `mk` the type `MyFoo<>`. Rule SUB-INHERIT-LOCAL reads the declaration of `MyFoo`, so `MyFoo<>` is a subtype of `Foo<X>`, where `X` is the type variable of `mk`. The type `MyFoo<>` does not say which type `X` stood for when the object was created. Since generic(`MyFoo`) = `MyFoo<>` has no type variables, rule CAST-DOWN accepts the type test `other instanceof MyFoo<>` from `Object<>`. Thus the call `a.<String>mk(a.<Integer>mk(o, i), s)` has type `Foo<String>`, but it returns a `Foo<Integer>`. This is the unsoundness of Java. It is there even if we keep the restriction of Java that the name of a local class can only be written inside the method declaring it, as `mk` does. Our formal system shows two more failure points. They are the same failure, shown more explicitly:

- Reduction does not preserve types, even without type tests. The call `a.<Integer>mk(o, i)` has type `Foo<Integer>`. It reduces to the value

```
new MyFoo<> extends Foo<Integer>(){ Integer x(){ return i; } }
```

The type of this value is `MyFoo<>`, a subtype of `Foo<X>` but not of `Foo<Integer>`. Java has no typing of values during reduction, so in Java this failure is invisible.

Typing depends on the names of type variables. Our system lets the name of a local class be written anywhere. Consider the methods

```
<Y> MyFoo2<> mk2(Y y){
  return new MyFoo2<> extends Foo<Y>(){ Y x(){ return y; } }; }
<Y> Foo<Y> leak(MyFoo2<> z){ return z; }
```

The method `leak` is well typed, since the type variable `Y` of `leak` has the same name as the type variable `Y` of `mk2`. Renaming `Y` into `Z` in `leak` makes `leak` ill typed. The call

```
a.<String>leak(a.<Integer>mk2(i))
```

has type `Foo<String>`, but it returns a `Foo<Integer>`, without type tests.

Funnelling solves the problem in all three forms. Under rule NEW-LOCAL, the object created by `mk` has type `MyFoo<X>`, written `new MyFoo<X funnel> extends Foo<X>(){..}`. Since generic(`MyFoo`) = `MyFoo<X>`, rule CAST-DOWN accepts the type test `instanceof MyFoo<X>` from `Foo<X>`, but not from `Object<>`. The call `a.<Integer>mk(o, i)` reduces to a value of type `MyFoo<Integer>`, a subtype of `Foo<Integer>`.

...

briefly note that Java allows extra, “fresh” generic parameters in local classes

call out that we’re allowed to declare return types that mention an inner class; inner classes are “public”, and our approach works even in this case, the more restrictive case of Java is naturally covered

articulate early: It’s not just about generic reification: just checking type parameters aren’t enough, as our MyFoo cases have no generic params. You need to *deeply* check reified generics, you need to check not just the base type but also check the choice(s) of parent types from the context into the runtime check.

$$
\begin{array}{rrcl}
\text{Values} & v & ::= & \texttt{new}\ C\texttt{<}Gs\texttt{>(}vs\texttt{)} \mid \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{)\{}Ms\texttt{\}} \\
\text{Eval. Ctx.} & \mathcal{E} & ::= & \square \mid \mathcal{E}\texttt{.<}Gs\texttt{>}m\texttt{(}es\texttt{)} \mid v\texttt{.<}Gs\texttt{>}m\texttt{(}vs\ \mathcal{E}\ es\texttt{)} \\
 & & & \mid \mathcal{E}\texttt{.[}G\texttt{]<}Gs\texttt{>}m\texttt{(}es\texttt{)} \mid v\texttt{.[}G\texttt{]<}Gs\texttt{>}m\texttt{(}vs\ \mathcal{E}\ es\texttt{)} \\
 & & & \mid \mathcal{E}\texttt{.}f \mid \texttt{new}\ C\texttt{<}Gs\texttt{>(}vs\ \mathcal{E}\ es\texttt{)} \\
 & & & \mid \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\ \mathcal{E}\ es\texttt{)\{}Ms\texttt{\}} \\
 & & & \mid \texttt{if (}\mathcal{E}\ \texttt{instanceof}\ G\ x\texttt{) \{}\ e_1\ \texttt{\} else \{}\ e_2\ \texttt{\}} \\
\text{Contexts} & \mathcal{C} & ::= & \square \mid \mathcal{C}\texttt{.<}Ts\texttt{>}m\texttt{(}es\texttt{)} \mid e\texttt{.<}Ts\texttt{>}m\texttt{(}es\ \mathcal{C}\ es'\texttt{)} \\
 & & & \mid \mathcal{C}\texttt{.[}T\texttt{]<}Ts\texttt{>}m\texttt{(}es\texttt{)} \mid e\texttt{.[}T\texttt{]<}Ts\texttt{>}m\texttt{(}es\ \mathcal{C}\ es'\texttt{)} \\
 & & & \mid \mathcal{C}\texttt{.}f \mid \texttt{new}\ C\texttt{<}Ts\texttt{>(}es\ \mathcal{C}\ es'\texttt{)} \\
 & & & \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\ \mathcal{C}\ es'\texttt{)\{}Ms\texttt{\}} \\
 & & & \mid \texttt{if (}\mathcal{C}\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\} else \{}\ e_2\ \texttt{\}} \\
 & & & \mid \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ \mathcal{C}\ \texttt{\} else \{}\ e_2\ \texttt{\}} \\
 & & & \mid \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\} else \{}\ \mathcal{C}\ \texttt{\}}
\end{array}
$$

Figure 13 Values and contexts

4 (Un)Soundness of Method Local Classes

With our language primitives in place, we now consider type safety of two variants of our language. The two variants differ only in the typing of local classes:

1. ζ(Java) types local classes with rule NEW-LOCAL-JAVA of Figure 12, as Java does;
2. ζ(funnelled) types local classes with rule NEW-LOCAL of Figure 6.

Both variants use the type test of Java: rule IF-INSTANCEOF accepts only checked casts (Figure 9), and at run time rules IFTHEN and IFELSE compare class names only. As shown in Section 3, ζ(Java) is unsound. We now show that ζ(funnelled) is sound.

5 Miscellaneous

//——— MARCO: (6th time) This is not about generic erasure In a language with reified generics, if the language still decides that MyFoo is not generic, no impact of those reified generics is going to be seen in the case (MyFoo)other MyFoo extends Foo<X> is 'as generic' as String extends Comparable<String> A cast to String, would not look in the generics of Comparable<String> A cast to MyFoo, would not look in the generics of Foo<X> If subtyping has (nominal-)'reflexitity' then String <: String (no need to look at the generics in Comparable<String>, available or not. A language with implicity 'type scope' that could conceptually add to the 'name' when types are identified in the type system could change this. Eg: scala foo.Bar is a type and foo.Bar != beer.Bar //existential types on constants/values Broken.<X>m.MyFoo //existential types on generic-instantiated methods //———- COLIN: This is ambiguous: with reified generics, there are two implementation strategies, which yield different behaviours: 1. MyFoo allocated as a Foo<String> gets a different class table from MyFoo allocated as a Foo<Integer>. In this case, the cast will fail 2. All MyFoo will get the same inherited class table, but keep a separate field with the choice of X for Foo<X>, used for inherited methods. In this case, there are *two possible cast implementations*: a. Cast checks only vtable, and cast succeeds. Arguably a compiler that "treats MyFoo as non-generic" could lean towards this b. Cast checks both vtable and the runtime representation of the type parameter of Foo. Arguably a compiler that preserves this for inherited methods from Foo to work correctly would be likely to check this (since it would be checked in a cast to Foo<Integer>).

//———-

Modern Java: the shallow/deep question (2a/2b) only arises for a language that checks casts at run time instead of requiring the checked-cast premise; under modern Java it does not arise.

This is a problem in other languages (cite cite) as shown in greater details in Rel Work.

Now, assume we have a readable small step reduction over a version of FJ rich enough to express the problem. We will assume: - Simple generics - A syntax like anonymous inner classes, but where we give them names. We can call them named inner classes. This imports the concept of method local inner classes within an expression based calculus. //our model will also need casts + instanceof

```
class A{ A onlyA()-> new A() }
class B{ B onlyB()-> new B() }
class Foo<X> extends Object{ X x() {return x();} }
public class Broken {
  <X> Foo<X> m(Object other, X x) ->
    if(other instanceof MyFoo)
```

```
      then ((MyFoo)other)
      else new MyFoo() extends Foo<X>{ X x(){ return x; } }//Bad
  }
main:
  new Broken().<B>m(new Broken().<A>m(new Object(),new A()),new B()).x().onlyB();
```

```
class Foo<X> extends Object{ X x() {return x();} }
public class Broken {
  <X> Foo<X> m(Object other, X x) ->
    if(other instanceof MyFoo<X>) //unchecked instanceof (ill typed in our core)?
      then ((MyFoo<X>)other)//unchecked cast (ill typed in our core)?
      else new MyFoo<X>() extends Foo<X>{ public X x(){ return x; } }//good
  }
```

Modern Java: answered by Figure 9: with `other : Object`, `other instanceof MyFoo<X>` is ill typed, because `X` is not determined by `Object` (as javac: `Object cannot be safely cast to MyFoo<X>`). With `other : Foo<X>` it is well typed and sound under erasure.

Note how funnelling purpose is to insert the generics. Without funnelling, MyFoo is a non generic type, so the question about generic reification does not even arise.

small step reduction

```
01:  new Broken().<B>m(new Broken().<A>m(new Object(),new A()),new B()).x().onlyB();
02:  new Broken().<B>m(
    if(new Object() instanceof MyFoo<X>)
      then (MyFoo<A>) new Object()
      else new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
      ,new B()).x().onlyB();
03:  new Broken().<B>m(
    if(false)
      then (MyFoo<A>) new Object()
      else new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
      ,new B()).x().onlyB();
04:  new Broken().<B>m(
       new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
      ,new B()).x().onlyB();
05:   (if(new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } } instanceof MyFoo<B
       then (MyFoo<A>) new MyFoo<A>() extends Foo<A>{ public A x()->new A() }
       else new MyFoo<B>() extends Foo<B>{ public B x(){ return new B(); } }).x().onlyB()
06a:  (if(false)//reified gen
       then (MyFoo<B>) new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
       else new MyFoo<B>() extends Foo<B>{ public B x(){ return new B(); } }).x().onlyB()
07a:  (new MyFoo<B>() extends Foo<B>{ public B x(){ return new B(); } }).x().onlyB()
08a:  new B().onlyB()
09a:  new B()//success and different semantic
06b:  (if(true)//Java
       then (MyFoo<B>) new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
       else new MyFoo<B>() extends Foo<B>{ public B x(){ return new B(); } }).x().onlyB()
07b:  ((MyFoo<B>) new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }).x().onlyB
08b:  (new MyFoo<A>() extends Foo<A>{ public A x()->new A() }).x().onlyB()
```

```
     //assuming cast and instanceof behave consistently (unchecked)
09b:  new A().x().onlyB()
10b:  new A().onlyB() //stuck!!
6c:  (if(err)//compile or at least run time
       then ((MyFoo<B>) new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
       else new MyFoo<B>() extends Foo<B>{ public B x(){ return new B(); } }).x().onlyB()
```

// Maybe above we want MyFoo:Object<A> and MyFoo:Object<B>?

```
public class Broken {
  static <X> Object m(Object other, final X x, Function<X,String> f){
    class MyFoo{ public X x(){ return x; } }
    if(!(other instanceof MyFoo)){ return new MyFoo(); }
    MyFoo local= (MyFoo)other;//cast pass with no warnings
    System.out.println("Pass␣with␣" + x);
    System.out.println("old␣one␣was␣"+f.apply(local.x()));//throws ClassCastException
    return local;
  }
  public static void main(String[] a) {
    Object r1= Broken.<Integer>m("anything",5,i->""+i);
    Broken.<String>m(r1,"Hi",s->s);
  }
}
```

```
public class Broken {
  static <X> Object m(Object other, final X x, Function<X,String> f){
    class MyFoo{ public X x(){ return x; } }
    if(!(other instanceof MyFoo)){ return new MyFoo(); }
    MyFoo local= (MyFoo)other;//cast pass with no warnings
    System.out.println("Pass␣with␣" + x);
    System.out.println("old␣one␣was␣"+f.apply(local.x()));//throws ClassCastException
    return local;
  }
  public static void main(String[] a) {
    Object r1= Broken.<Integer>m("anything",5,i->""+i);
    Broken.<String>m(r1,"Hi",s->s);
  }
}
```

```
public class Broken {
  static <X> Object m(Object other, final X x, Function<X,String> f){
    class MyFoo{ public X x(){ return x; } }
    if(other instanceof MyFoo local){ //new instanceof+introduce var (pass with no warnings)
      System.out.println("Pass␣with␣" + x); //snekly 'local' is MyFoo<Integer> even when this is Brokem.<St
      System.out.println("old␣one␣was␣"+f.apply(local.x()));//throws ClassCastException
     }
  return new MyFoo();//snekly it is MyFoo<Integer>
  }
  public static void main(String[] a) {
    Object r1= Broken.<Integer>m("anything",5,i->""+i);
    Broken.<String>m(r1,"Hi",s->s);
  }
}
```

// TODO: could talk about doing this with modern Java type-switch

```
// Compiles without error, fails, on jdk 25.0.1
import java.util.function.Function;
class Broken {
  static <X> Object m(Object other, final X x, Function<X,String> f){
    class MyFoo{ public X x(){ return x; } }
    switch (other) {
        case MyFoo local: // implicit unchecked cast
            System.out.println("Pass␣with␣" + x); //snekly `local` is MyFoo<Integer> even when this is B
            System.out.println("old␣one␣was␣"+f.apply(local.x()));//throws ClassCastException (on the re
            break;
        default:
            break;
    }
    if(other instanceof MyFoo local){ //new instanceof+introduce var (pass with no warnings)
      System.out.println("Pass␣with␣" + x); //snekly `local` is MyFoo<Integer> even when this is Brokem.
      System.out.println("old␣one␣was␣"+f.apply(local.x()));//throws ClassCastException
     }
  return new MyFoo();//snekly it is MyFoo<Integer>
  }
  public static void main(String[] a) {
    Object r1= Broken.<Integer>m("anything",5,i->""+i);
    Broken.<String>m(r1,"Hi",s->(s.length())+""); //Broken.<String>m(r1,"Hi",s->s);
  }
}
```

```
//Example that breaks in a NEW WAY in the bytecode
// https://godbolt.org/z/sMnxPG1TE
import java.util.function.Function;
class Broken {
  static <X> Object m(Object other, final X x, Function<X,Integer> f){
    class MyFoo{ public X x(){ return x; } }
    switch (other) {
        case MyFoo local:
            System.out.println("Pass␣with␣" + x); //snekly `local` is MyFoo<Integer> even when this is B
            System.out.println("old␣one␣was␣"+f.apply(local.x()));//throws ClassCastException
            break;
        default:
            break;
    }
    if(other instanceof MyFoo local){ //new instanceof+introduce var (pass with no warnings)
      System.out.println("Pass␣with␣" + x); //snekly `local` is MyFoo<Integer> even when this is Brokem.
      System.out.println("old␣one␣was␣"+f.apply(local.x()));//throws ClassCastException
     }
  return new MyFoo();//snekly it is MyFoo<Integer>
  }
  public static void main(String[] a) {
    Object r1= Broken.<Integer>m("anything",5,i->i);
    Broken.<String>m(r1,"Hi",s->s.length());
  }
}
// Above seems to fail but without explicit checkcast, failing one of the virtual method calls
```

With investiagion in bytecode and trying both s->s.length() and String::length we think that in order to save on bytecode size, our JVM enviroment under test adds casts in all lambda

calls so that meth refecences lambda calls do not need specialized bytecode (eg MyInterface a=String::length)

Marco had this idea that casts may be avoided by using dynamic dispatch returning this. Can not work, since Meth local class is private, there is no way an external interface can mention it to provide the cast method. Of course, the method local is private to the body (this means also the method signature can not use it) This means no self:Foo method, nor self:T method of a generic class eventually instantiating T=Foo.

This means the core language DO NEED if(e instanceof T x) else

//nested funnelling: new A<X>() B<X> f()-> new B<X>() new A<X>() <K>B<X,K> f()-> new B<X,K>()

using 'extends' for declarations in top level classes and generic methods using 'funnel' for named inner classes new A<X funnel Object>() B<X> f()-> new B<X funnel Object>() new A<X funnel Object>() <K extends Object> B<X,K> f()-> new B<X funnel Object,K funnel Object>() or, if the bounds are not repeated new A<X funnel>() <K extends Object> B<X,K> f()-> new B<X funnel,K funnel>() Very funky syntax: we get new B<X funnel Object,K funnel Object>() but, during reduction, this can become we get new B<Person,K funnel Object>() where we have a mixture of concrete types and generic funnelling

Global class definition uses class C<X extends Bar>.. named method local class uses new C<X funnel>().. normal new: new C<Ts>()

```
e ::= x | e.m(es) | e.f | new C<Ts>(es) | new C<Fs>(){Ms} | if (e0 instanceof T x){ e1 }else{ e2
T ::= C<Ts> | X
F ::= C<CCs> | X funnel// Important, can not be T | X funnel, otherwise we could have an X not 'f
CC::= C<CCs> //types with zero free type variables
B::=  X extends T // Need T, not F, to be able to do things like (Java) class Bar<X,Y extends X>
Dec::= class C<Bs>{ Ms Ss} //assume standard constructor
S ::= T x; //field/slot
M ::= Sig; | Sig{ return e; }
Sig::= <Bs> T m(x1:T1,..xn:Tn)
G ::= x1:T1..xn:Tn
```

//we do not need Object to be able to write 'extends Object' //this is exactly as the distinction of checked and unchecked exceptions: formally it evaporates in syntacitc sugar where all is checked //and all methods 'throws Error, RTE' // Can assume the user provides their own Object type Note: an use of X is just X, a declaration of X is X extends T; a funnelling of X is X funnel Program is just many Dec plus an e typable in empty MARCO TODO: typechecking Decs;Bs;G|-e:T Decs|-Dec:ok Decs;T|-M:ok e –> e' Note, there are multiple semantics/ts: e instanceof CC //always clear e instanceof T with T not of form CC would not typecheck

e –> e' for Java with warn->err: just have CC in instanceof Java with magic reifined cast: just have T in instanceof (since reduction will replace(thus remember) generics Java with warn->off: AT RUN TIME instanceof only checks base class, ignores generic parameters AT COMPLILE TIME, instanceof can be err if the full type (including generics) is not potentially correlated)?? unobvious consider interfaces In prose, should explain that we won't flag cast of some type Foo to some type Bar, even though Java sometimes flags unrelated casts

Modern Java: these three semantics are replaced by the single one of Java 16+: erased run-time check, plus the checked-cast premise of rule IF-INSTANCEOF (Figure 9). Unrelated types are rejected by it, as in Java.

<X extends List<Y funnel»
class Foo<X> class Getter<X funnel> <Y extends List<X» X getFirst(Y y) y.getOrElse(0,null)

<Y> .. return foo<Y,MyX>(); <Y,X extends List<Y» foo() return Foo<X funnel>()...
X ;

6 Related Work

It is not uncommon to have "intentional unsoundness" in industrial languages, such as Java's covariant array subtyping, or TypeScript's covariant method subtyping. However these are intentionally designed into the language to improve ease of use, and in most cases (like Java's array handling) problematic uses are detected at runtime (e.g., Java checks upon storing into an array that the new entry value is a subtype of the array's original content type).

However, unintentional unsoundness in industrial type systems is rarer. The most notable example is Amin and Tate's discovery [2] that Java and Scala's handling of generic type constraints was rendered unsound by the fact that `null` is treated as inhabiting all reference types, making all type constraints (even nonsensical ones) satisfiable. Servetto et al. [6] first pointed out the issue publicly, though in the Q&A session after the talk, multiple Java designers were unsurprised — as Gilad Bracha put it, "Once I saw the title of your talk, I understood what the problem was."

Colin says: As I listened to the Q&A session again, James framed this as a question of whether two instances allocated in different contexts should be considered the same or not. I think that might be useful framing to bring up early here. Gilad also pointed out that (paraphrasing) the BETA people knew an inner class was a property of the instance, not its own class.

discuss C# casts with reified generics

The key issues at work in the interaction of method-local inner classes and generics arise from the interaction of (1) nominal dynamic downcasts, (2) design choices about what runtime information about generic parameters is both persisted *and checked* by casts, and (3) (not) tracking all dependencies on type parameters in the type system. Essentially, Java and Scala both treat `MyFoo` as a sort of existential type — that there exists some `X` such that `MyFoo` extends `Foo<X>`. However, treating `MyFoo` as a closed rather than generic type is equivalent to assuming that there is only one existential for all instances — associated with the declaration site — rather than potentially different existentials for different instances. This combined with the fact that the JVM is unable to dynamically check for different instantiations leads to the unsoundness. Our dynamic solution resolves this with a specific implementation of casts on reified generics.

Modern Java: under modern Java there is no separate dynamic solution; see the note on the dynamic approach in the introduction.

Our static solution effectively prohibits the de facto existential capture of the enclosing type parameter.

The decision to treat types like the `MyFoo` variants in our examples as closed existential types that depend on other choices of types is not without parallels in other languages. Haskell and Rust, for example, include associated types [3], where a typeclass has a type as a member. A yet closer analogue is Scala's path-dependent types , where types may be *members* of an individual object, rather than associated with an individual class. The code below adapts the troublesome example method using Scala's type members:

cite dot

```
trait Foo { type X; def x() : X; }
def m[Y](other: Any, newx: Y) : Foo2 =
    class MyFoo extends Foo { type X = Y; override def x() : X = newx; }
    newx match {
        case f: MyFoo => f
        case _ => MyFoo()
    }
```

This code compiles without warnings — and is not an issue. In this case the dependency of `MyFoo` instances on the type parameter used to call `m` is made explicit with a type member. But Scala now treats thes

References

1 Ole Agesen, Stephen N Freund, and John C Mitchell. Adding type parameterization to the java language. In *Proceedings of the 12th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*, pages 49–65, 1997.

2 Nada Amin and Ross Tate. Java and scala's type systems are unsound: The existential crisis of null pointers. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2016, page 838–848, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2983990.2984004.

3 Manuel MT Chakravarty, Gabriele Keller, Simon Peyton Jones, and Simon Marlow. Associated types with class. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 1–13, 2005.

4 Andrew Kennedy and Don Syme. Design and implementation of generics for the. net common language runtime. In *Proceedings of the ACM SIGPLAN 2001 conference on Programming language design and implementation*, pages 1–12, 2001.

5 Martin Odersky, Enno Runne, and Philip Wadler. Two ways to bake your pizza—translating parameterised types into java. In *Generic Programming: International Seminar on Generic Programming Dagstuhl Castle, Germany, April 27–May 1, 1998 Selected Papers*, pages 114–132. Springer, 2001.

6 Marco Servetto, Nick Webster, and Colin S. Gordon. Java method-local inner classes are unsound. Talk at UNSOUND 2024, co-located with SPLASH 2024, October 2024. Recording at https://www.youtube.com/watch?v=DfkOpk733dQ. URL: https://2024.splashcon.org/details/unsound-2024-papers/3/Java-Method-Local-Inner-Classes-are-Unsound.